COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

23IT723 - DEVOPS LABORATORY
PROJECT
EMPLOYEE LEAVE MANAGEMENT SYSTEM



OCTOBER 2026

Submitted by

2303717620521009

BALAKUMARAN M

COIMBATORE INSTITUTE OF TECHNOLOGY
(Government Aided Autonomous Institution Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled "**EMPLOYEE LEAVE MANAGEMENT SYSTEM WITH FULL-STACK DEVOPS CI/CD PIPELINE**" is the bonafide work of 2303717620521009 – BALAKUMARAN M completed during the **academic year 2025-2026 - Semester VII** for the project presentation of **23IT723 - DEVOPS LABORATORY** under our supervision.

Certified that the candidate was examined by using the project work examination.

Review Members:

1. **Dr.M.Sangeetha**
2. **Dr.E.Arul**

23IT723 - DEVOPS LABORATORY

FINAL MINI PROJECT - EVALUATION SCHEME

Final Mini Project Assessment

The final mini project shall require students to integrate the major DevOps practices covered in the laboratory. The project should demonstrate practical implementation of version control, collaborative development, Continuous Integration, containerization and/or automated configuration/provisioning, with proper execution evidence.

S.No.	Evaluation Component	Key Verification Points	Marks
1	Problem Definition & Project Planning	Problem statement, objectives, requirements, scope, module identification, technology selection and workflow planning.	10
2	Git Repository & Version Control	Repository creation, meaningful commits, README, .gitignore, remote repository, push/pull/fetch operations and commit history.	15
3	Branching, Merging & Collaboration	Feature branches, parallel development, merging, conflict handling, branch organization and collaborative workflow.	10
4	Application Development & Build	Working application/source code, project structure, dependency management, build configuration and successful execution.	15
5	Jenkins Continuous Integration	Jenkins configuration, source-code integration, automated checkout, build job/pipeline, build execution and console verification.	15
6	Docker / Containerization	Dockerfile, image creation, container execution, port/configuration handling, application deployment and container verification.	10
7	Automation / Configuration Management	Ansible/Chef/Puppet/SaltStack or equivalent automation where applicable; repeatable configuration/provisioning and successful execution.	5

S.No.	Evaluation Component	Key Verification Points	Marks
8	Testing & Verification	Functional testing, build verification, error identification, test evidence and corrective action.	5
9	Documentation & Technical Presentation	Architecture/workflow diagram, commands/configuration, screenshots, results, troubleshooting, conclusion and repository details.	5
10	Final Demonstration & Viva-Voce	Live demonstration, explanation of implementation, troubleshooting responses and technical understanding.	10
		TOTAL	**100**

Recommended Mark Distribution within the Final Demonstration

For uniform evaluation, the examiner may use the following internal split for the implementation and demonstration components.

Assessment Area	Marks	Awarded Marks	Remarks
Working implementation and correctness	30		
DevOps tool integration and automation	20		
Version control and collaborative workflow	15		
Build / deployment / container verification	15		
Testing and troubleshooting	5		
Documentation and technical presentation	5		
Live demonstration and viva-voce	10		
TOTAL	**100**		

Minimum Evidence to be Submitted

- Project repository URL and project README.
- Git commit history and branch/merge evidence.
- Jenkins job/pipeline configuration and successful build evidence.
- Dockerfile, image and running container evidence where Docker is used..
- Architecture/workflow diagram.
- Short technical report containing implementation steps, results and troubleshooting.
- Live demonstration of the complete DevOps workflow.

Evaluation Guidelines

- Marks shall be awarded primarily for a working implementation and demonstrable output.
- Copied source code without functional understanding shall not receive full marks.
- Screenshots may support the evaluation, but critical components should be demonstrated live wherever possible.
- The final viva shall focus on the student's own implementation, commands used, tool configuration and troubleshooting.

Faculty Evaluation Record

Student Name		**Register No.**	
Project Title		**Date**	
Final Mark	**/ 100**		
Faculty Evaluator I	**Dr.M.Sangeetha.**	**Signature**	
Faculty Evaluator II	**Dr.E.Arul.**	**Signature**	

TABLE OF CONTENTS

S. NO.	CONTENT	PAGE NO.
1	INTRODUCTION	06
2	OBJECTIVES	06
3	ABSTRACT	08
4	TECHNOLOGY STACK	09
5	SYSTEM ARCHITECTURE	10
6	MODULE 1 — USER INTERFACE & LEAVE PORTAL	11
7	MODULE 2 — EXPRESS BACKEND & LEAVE MANAGEMENT API	11
8	MODULE 3 — AUTHENTICATION & ROLE-BASED ACCESS CONTROL	12
9	MODULE 4 — DATABASE MANAGEMENT	13
10	MODULE 5 — GIT & GITHUB SOURCE CONTROL	15
11	MODULE 6 — DOCKER CONTAINERIZATION	16
12	MODULE 7 — DOCKER COMPOSE ORCHESTRATION	17
13	MODULE 8 — JENKINS CI/CD PIPELINE	18
14	MODULE 9 — AUTOMATED TESTING	19
15	MODULE 10 — SONARQUBE CODE QUALITY	20
16	MODULE 11 — MONITORING WITH PROMETHEUS & GRAFANA	21
17	MODULE 12 — APPLICATION & DEPLOYMENT VERIFICATION	22
18	API ENDPOINTS	23
19	RESULTS AND OUTPUT	25
20	COMMANDS USED	26
21	ADVANTAGES	27
22	LIMITATIONS	27
23	FUTURE ENHANCEMENTS	27
24	CONCLUSION	27
25	REFERENCES	28

1. INTRODUCTION

The Employee Leave Management System is a web-based application developed to simplify how organisations request, approve and track employee leave. Employees log in, view their leave balances across five leave categories, and submit leave requests. HR managers review and approve or reject requests with remarks, while administrators manage employee accounts, leave quotas and company-wide analytics.

The application is built with Node.js and Express on the backend, a vanilla HTML5, CSS3 and JavaScript single page frontend, and a database layer that works with SQLite during development and testing and MySQL 8.0 in production. Authentication uses JSON Web Tokens (JWT) with role-based access control for the employee, manager and admin roles.

The project also implements a complete DevOps workflow around the application. Git and GitHub provide source control, Jenkins runs a seven-stage declarative CI pipeline with an automated test quality gate, Docker and Docker Compose package and orchestrate the application with MySQL, Prometheus and Grafana, SonarQube is configured for static code analysis, and Prometheus with Grafana provides runtime monitoring through custom application metrics.

2. OBJECTIVES

- To develop a web-based leave management system in which employees can view balances, submit, track and cancel leave requests.
- To provide an HR/manager portal for reviewing, filtering, approving and rejecting leave requests with remarks.
- To provide an admin portal for managing employee accounts, leave quotas and company-wide leave analytics.
- To secure the application using JWT authentication, bcrypt password hashing and role-based access control.
- To support SQLite for development/testing and MySQL for production through a single database abstraction layer.
- To manage the source code using Git and GitHub, with the Jenkinsfile maintained as code in the repository.
- To automate build verification and testing through a seven-stage Jenkins pipeline in which failed tests stop the pipeline.
- To containerize the application using Docker and orchestrate the application, MySQL, Prometheus and Grafana using Docker Compose.
- To validate the application using 17 automated Jest and Supertest test cases.
- To monitor application health and behaviour using health checks, Prometheus custom metrics and Grafana dashboards, and to configure SonarQube for code quality analysis.

3. ABSTRACT

The Employee Leave Management System is a full-stack web application combined with a complete DevOps CI/CD workflow. Conventional leave management using paper forms, spreadsheets and email approvals is slow, lacks an audit trail and gives no real-time visibility, while manual deployment and the absence of automated testing and monitoring reduce software reliability. The proposed system provides role-based portals for employees, HR managers and administrators, with real-time leave balances for five leave categories (Casual, Sick, Earned, Maternity/Paternity and Bereavement), automatic day calculation, and an approval workflow with remarks.

The backend is developed using Node.js 20 and Express.js with JWT authentication and bcrypt password hashing. A database abstraction layer allows the same code to run on SQLite for development and testing and on MySQL 8.0 for production. The project implements 18 REST API endpoints, a /health probe and a Prometheus /metrics endpoint.

The DevOps workflow uses Git and GitHub for version control, Jenkins for a seven-stage declarative CI pipeline, Jest and Supertest for 17 automated test cases, Docker for a non-root Alpine-based image with a built-in health check, Docker Compose for orchestrating the application with MySQL, Prometheus and Grafana, and SonarQube for static code analysis configuration. The complete workflow follows **GitHub → Jenkins → Install → Test (Quality Gate) → Build Verification → Health Check → Docker Compose → Express → MySQL → Prometheus → Grafana**. All 17 automated tests pass, demonstrating the integration of a functional web application with modern DevOps practices.

4. TECHNOLOGY STACK

Category	Technology	Version / Purpose
Runtime	Node.js	v20 LTS
Backend Framework	Express.js	^4.21.2
Authentication	jsonwebtoken (JWT)	^9.0.2
Password Hashing	bcryptjs	^2.4.3 (cost factor 10)
Database (Development)	SQLite	Node.js built-in node:sqlite
Database (Production)	MySQL 8.0 with mysql2 driver	^3.12.0
Monitoring Library	prom-client	^15.1.3
Testing Framework	Jest	^29.7.0
HTTP Test Client	Supertest	^7.0.0
Test Reporter	jest-junit	^17.0.0 (junit.xml for Jenkins)

Category	Technology	Version / Purpose
Environment Config	dotenv	^16.4.7
Cross-Origin	cors	^2.8.5
Frontend	HTML5, CSS3, Vanilla JavaScript	Single Page Application
Version Control	Git and GitHub	Source code repository
CI/CD Server	Jenkins	Latest LTS
Containerization	Docker	Latest
Orchestration	Docker Compose	File format v3.8
Code Quality	SonarQube Scanner	Latest
Metrics Storage	Prometheus	Latest
Dashboards	Grafana	Latest

5. SYSTEM ARCHITECTURE

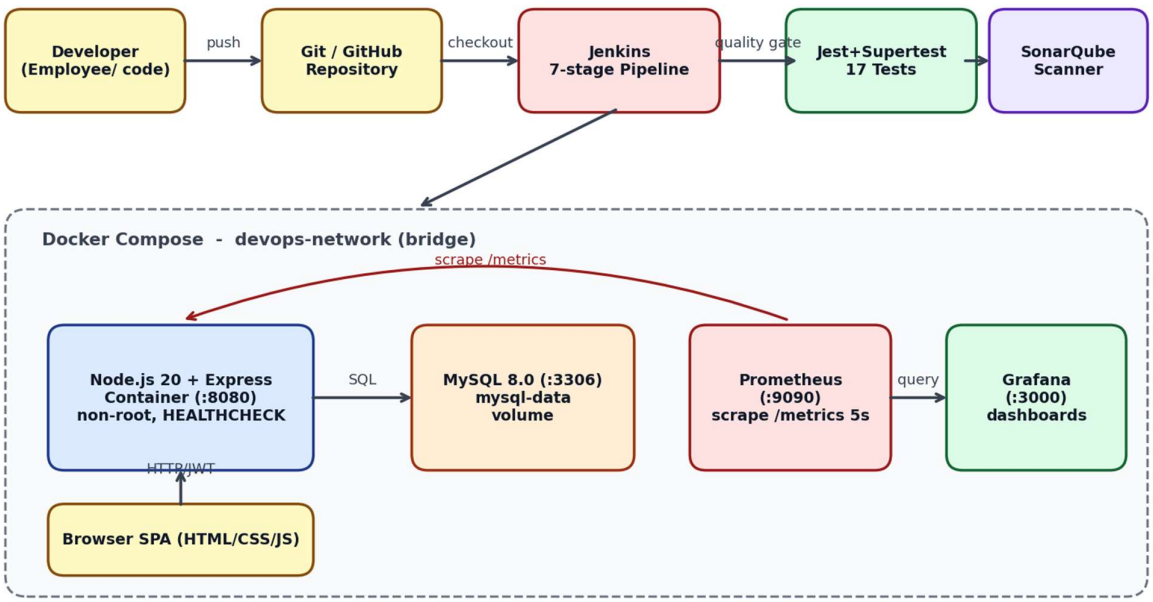


Figure 5.1: System Architecture and DevOps Workflow

1. Development: The developer creates and updates the application in the Employee project directory.
2. Source Code Management: Source code, Dockerfile, Jenkinsfile and configuration files are maintained using Git.
3. GitHub Repository: Code is pushed to the remote GitHub repository for centralized management.

4. Jenkins Checkout: Jenkins retrieves the latest code using checkout scm and starts the pipeline defined in the Jenkinsfile.
5. Dependency Installation: The pipeline installs all Node.js packages using npm install.
6. Automated Testing: Jest runs 17 test cases; if any test fails, the pipeline stops and nothing proceeds (quality gate).
7. Verification Stages: Project structure, application build/database initialisation and the /health endpoint are verified.
8. Containerization: The Dockerfile packages the application as a non-root Alpine image with a built-in health check.
9. Orchestration: Docker Compose starts the application, MySQL, Prometheus and Grafana on a shared bridge network with persistent volumes.
10. Monitoring: Prometheus scrapes /metrics every 5 seconds and Grafana visualises the data.
11. User Interaction: Employees, managers and administrators use the web interface through the browser.

MODULE 1 — USER INTERFACE & LEAVE PORTAL

This module provides the browser-based interface of the application. It is a Single Page Application in which all views are loaded dynamically through JavaScript without page reloads, using a glassmorphism dark-mode design.

Main Functions

- Provides login using email and password.
- Displays leave balance cards showing total, used and remaining days for each of the five leave categories.
- Provides a leave application modal with leave type, date pickers and reason.
- Displays the user's leave requests in a table with colour-coded PENDING, APPROVED and REJECTED badges.
- Allows employees to cancel their own pending requests.
- Provides a manager approval/rejection modal with a remarks field.
- Provides an admin employee directory with create, update and delete forms.
- Provides a Live Lab Demo Hub ribbon for one-click role switching using /api/auth/demo-users.
- Provides a DevOps telemetry panel with links to /health and /metrics.

Frontend Files

```
src/public/index.html
src/public/css/style.css
src/public/js/app.js
```

Technologies: HTML5, CSS3, JavaScript.

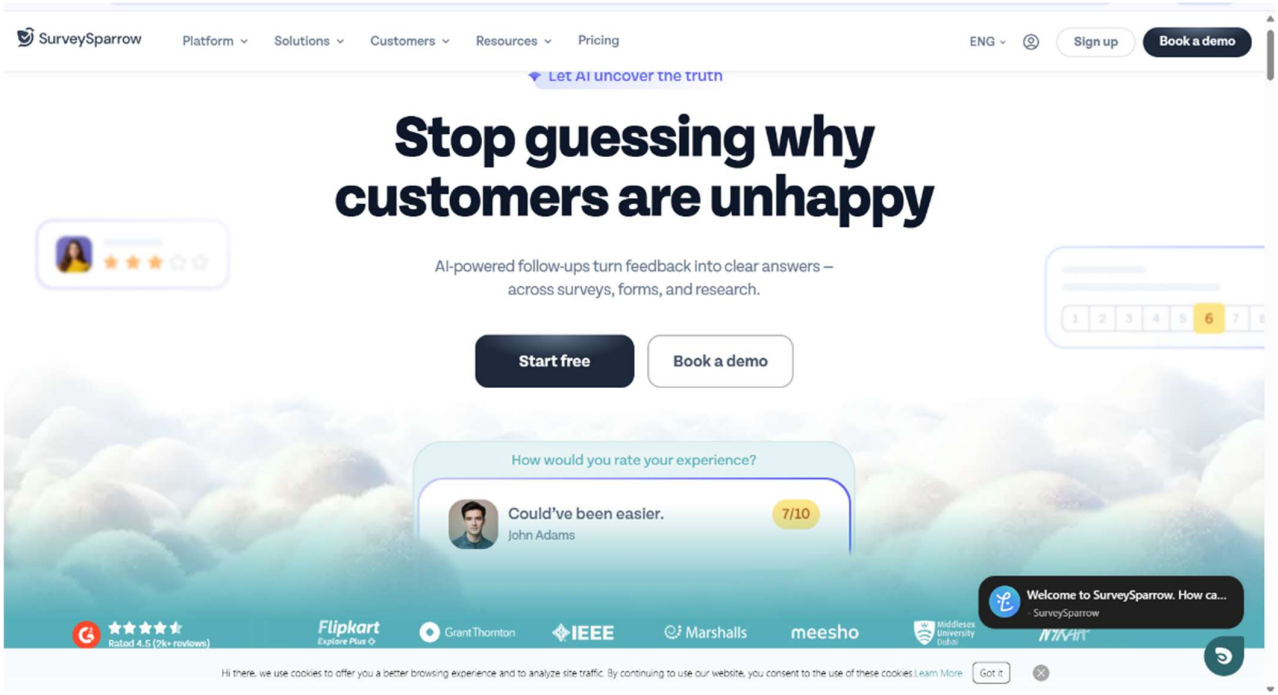


Figure 7.1: Login Page and Live Lab Demo Hub

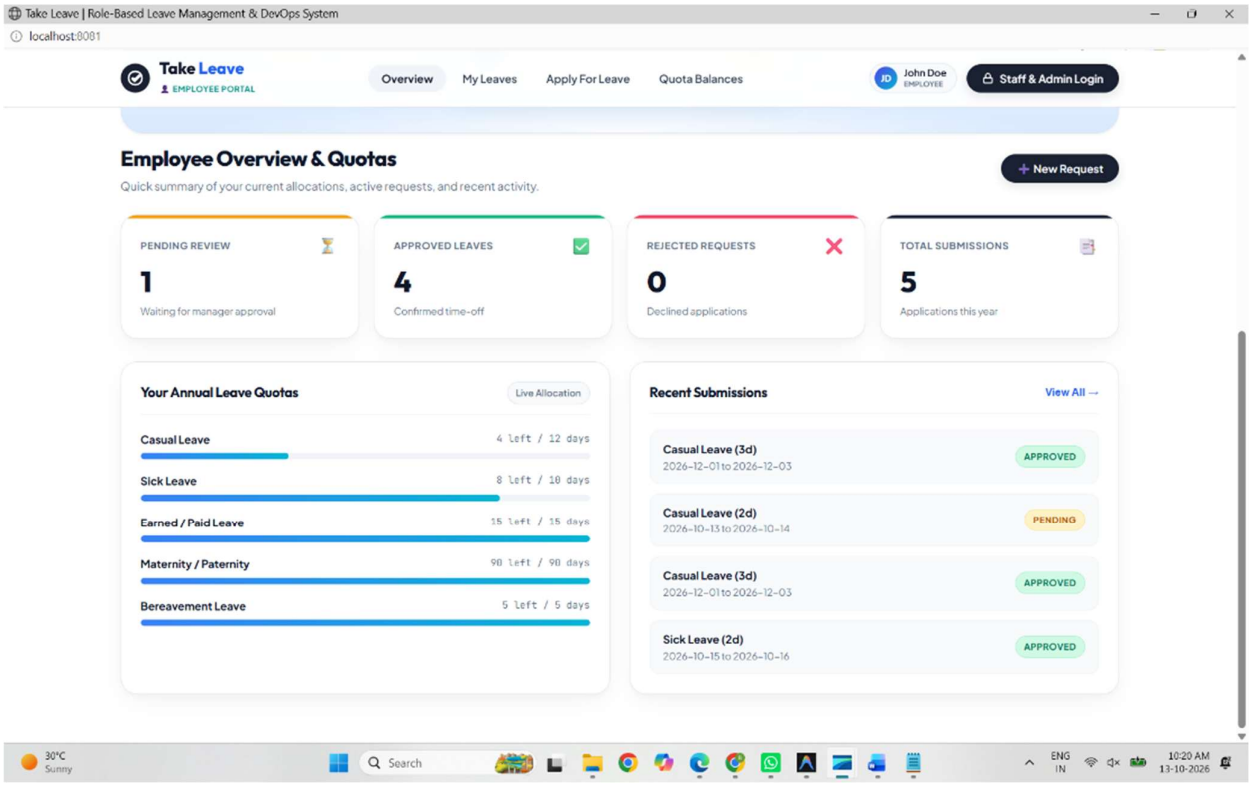


Figure 7.2: Employee Dashboard with Leave Balance Cards

MODULE 2 — EXPRESS BACKEND & LEAVE MANAGEMENT API

This module contains the application logic and REST API. The Express app is created by an app factory in app.js, while server.js starts the HTTP server with automatic port fallback and graceful shutdown.

Main Functions

- Exposes 18 REST API endpoints for authentication, employees, leaves, leave types, statistics, health and metrics.
- Auto-calculates the number of leave days (inclusive of both dates).
- Validates date ranges and rejects invalid requests with HTTP 400.
- Manages the leave lifecycle: PENDING to APPROVED or REJECTED, with manager remarks.
- Allows employees to cancel their own pending requests.
- Falls back automatically to the next free port (8081, 8082, ...) if the configured port is in use.
- Handles SIGTERM and SIGINT by closing the server and database connections and exiting with code 0.
- Serves the static frontend files from src/public.

Backend Structure

```
src/app.js         - Express app factory
src/server.js      - HTTP server, port fallback, graceful shutdown
src/config/        - environment-based configuration
src/routes/        - auth, employees, leaves, leaveTypes, stats
src/middleware/    - auth.js (JWT, RBAC), metrics.js (Prometheus)
```

Technology: Node.js with Express.js.

C: > GitLab > Employee > src > JS app.js

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const { register, metricsMiddleware } = require('./middleware/metrics');
const db = require('./db');

const authRoutes = require('./routes/auth');
const employeeRoutes = require('./routes/employees');
const leaveRoutes = require('./routes/leaves');
const leaveTypeRoutes = require('./routes/leaveTypes');
const statsRoutes = require('./routes/stats');

const app = express();

// Middlewares
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Track HTTP metrics for Prometheus
app.use(metricsMiddleware);

// Serve Static Frontend Assets
app.use(express.static(path.join(__dirname, 'public')));

// DevOps Health Check endpoint (for Docker / Kubernetes probes)
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'UP',
    timestamp: new Date().toISOString(),
    uptime: Math.floor(process.uptime()),
    database: db.getDbInfo(),
    service: 'employee-leave-management'
  });
});

// DevOps Prometheus Metrics endpoint
app.get('/metrics', async (req, res) => {
  try {
    res.set('Content-Type', register.contentType);
```

```
app.get('/metrics', async (req, res) => {
  try {
    res.set('Content-Type', register.contentType);
    res.end(await register.metrics());
  } catch (err) {
    res.status(500).end(err);
  }
});

// API Routes
app.use('/api/auth', authRoutes);
app.use('/api/employees', employeeRoutes);
app.use('/api/leaves', leaveRoutes);
app.use('/api/leave-types', leaveTypeRoutes);
app.use('/api/stats', statsRoutes);

// SPA fallback for HTML frontend
app.get('*', (req, res, next) => {
  if (req.path.startsWith('/api') || req.path === '/health' || req.path === '/metrics') {
    return next();
  }
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

// Global Error Handler
app.use((err, req, res, next) => {
  console.error('Unhandled application error:', err);
  res.status(500).json({
    success: false,
    message: 'Internal server error occurred.'
  });
});

module.exports = app;
```

Figure 7.4: Backend Source Structure in the Editor

MODULE 3 — AUTHENTICATION & ROLE-BASED ACCESS CONTROL

This module secures the application. Users authenticate with email and password and receive a JWT, which is validated on every protected route.

Main Functions

- Validates credentials using bcrypt.compareSync() against the stored password hash.
- Signs a JWT containing employee_id, name, email, role and department, valid for 8 hours.
- The authenticate middleware verifies the Bearer token on every protected route.
- The requireRole() middleware compares the user's role with the allowed roles and returns HTTP 403 Forbidden otherwise.
- Only managers and admins can approve or reject leaves; only admins can create, update or delete employees.
- Passwords are hashed with bcrypt (cost factor 10); the container runs as the non-root node user.

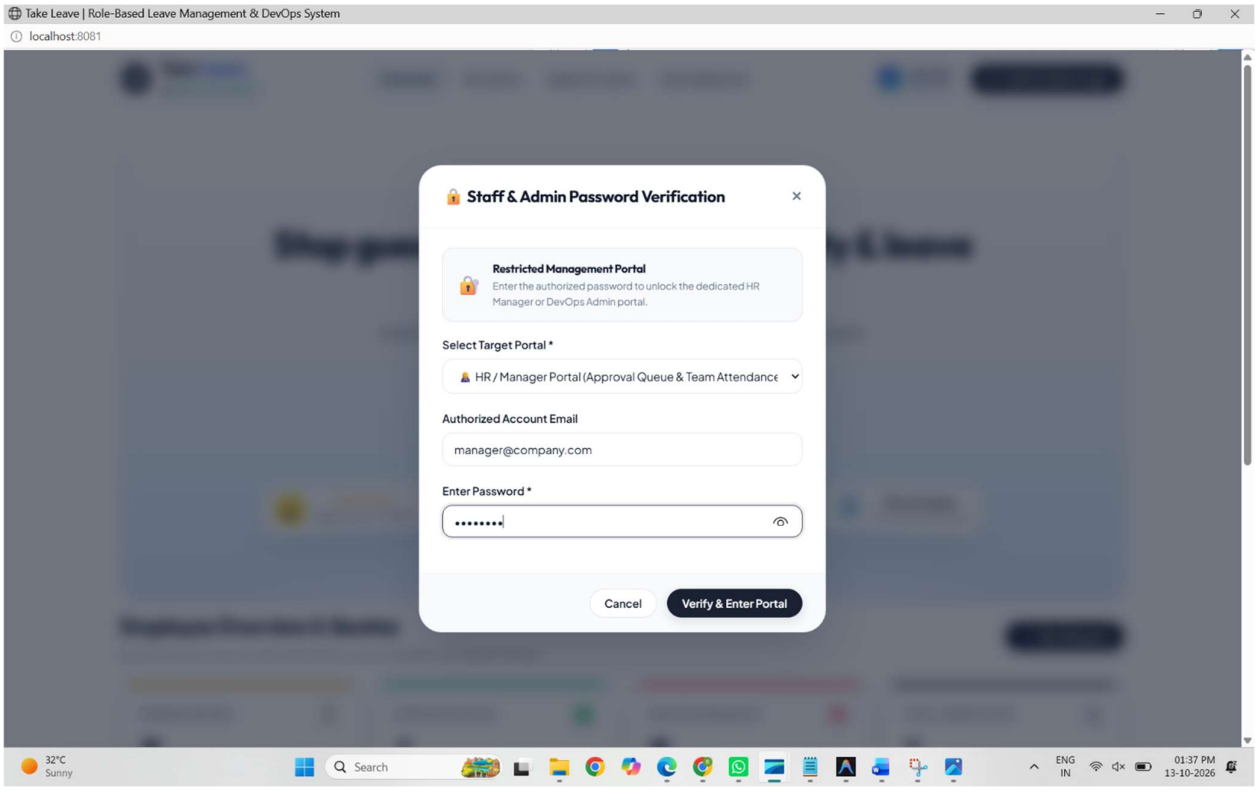


Figure 7.5: Successful Login and JWT Response

MODULE 4 — DATABASE MANAGEMENT

This module manages data storage. A database abstraction layer (src/db/index.js) exposes query() and execute() functions that work identically on SQLite and MySQL, so route handlers do not depend on the database engine. The application auto-detects MySQL and silently falls back to SQLite, so the same code runs in CI and in production.

Main Functions

- Creates the schema separately for SQLite and MySQL (schema.js).
- Seeds default employees, leave types and sample leave requests (seed.js, init.sql).
- Uses SQLite for development and testing, requiring no external database.
- Uses MySQL 8.0 in Docker Compose with persistent storage in the mysql-data volume.
- Maintains relationships using foreign keys with cascade delete.
- **Table 1: employees**
- employee_id: INT AUTO_INCREMENT, Primary Key
- name: VARCHAR(100), full name
- email: VARCHAR(100) UNIQUE, login email
- password: VARCHAR(255), bcrypt hashed (cost 10)
- department: VARCHAR(50), e.g. Engineering, HR, IT
- role: VARCHAR(20), employee, manager or admin
- created_at: TIMESTAMP, set automatically
- **Table 2: leave_types**
- leave_type_id: INT AUTO_INCREMENT, Primary Key
- type_name: VARCHAR(50) UNIQUE, e.g. Casual Leave
- available_days: INT, annual quota (default 12)
- description: VARCHAR(255), human readable description
-

```
-- Seed Data (Passwords bcrypt-hashed with 'admin123', 'manager123', 'password123')
INSERT IGNORE INTO employees (employee_id, name, email, password, department, role) VALUES
(1, 'DevOps Admin', 'admin@company.com', '$2a$10$wT5g80nOq9d9V9R1d2L8v.w6M0F2H3P.4vQ/O5I1E7sK5G6G0M0Fa', 'DevOps & IT', 'admin'),
(2, 'Sarah Connor (HR Manager)', 'manager@company.com', '$2a$10$wT5g80nOq9d9V9R1d2L8v.w6M0F2H3P.4vQ/O5I1E7sK5G6G0M0Fa', 'Human Resources', 'manager'),
(3, 'John Doe', 'john.doe@company.com', '$2a$10$wT5g80nOq9d9V9R1d2L8v.w6M0F2H3P.4vQ/O5I1E7sK5G6G0M0Fa', 'Engineering', 'employee'),
(4, 'Jane Smith', 'jane.smith@company.com', '$2a$10$wT5g80nOq9d9V9R1d2L8v.w6M0F2H3P.4vQ/O5I1E7sK5G6G0M0Fa', 'Marketing', 'employee'),
(5, 'Alex Kumar', 'alex.kumar@company.com', '$2a$10$wT5g80nOq9d9V9R1d2L8v.w6M0F2H3P.4vQ/O5I1E7sK5G6G0M0Fa', 'Product Design', 'employee');

INSERT IGNORE INTO leave_types (leave_type_id, type_name, available_days, description) VALUES
(1, 'Casual Leave', 12, 'General purpose personal time off'),
(2, 'Sick Leave', 10, 'Medical and health related leave'),
(3, 'Earned / Paid Leave', 15, 'Annual accrued vacation leave'),
(4, 'Maternity / Paternity', 90, 'Parental bonding and family leave'),
(5, 'Bereavement Leave', 5, 'Compassionate leave for family loss');

INSERT IGNORE INTO leave_requests (leave_id, employee_id, leave_type_id, from_date, to_date, days_requested, reason, status, manager_remarks) VALUES
(1, 3, 2, '2026-10-15', '2026-10-16', 2, 'Seasonal viral fever doctor recommended rest', 'APPROVED', 'Approved. Get well soon!'),
(2, 4, 1, '2026-10-20', '2026-10-22', 3, 'Attending family wedding anniversary celebration', 'PENDING', NULL),
(3, 5, 3, '2026-11-01', '2026-11-05', 5, 'Planned vacation trip to Himalayas', 'PENDING', NULL),
(4, 3, 1, '2026-09-10', '2026-09-11', 2, 'Personal home relocation work', 'APPROVED', 'Approved by Sarah');
```

Figure 7.7: MySQL Tables and Seeded Data

MODULE 5 — GIT & GITHUB SOURCE CONTROL

This module manages the source code and development history. Git is used locally and GitHub is the remote repository from which Jenkins retrieves the code. The Jenkinsfile is stored in the repository (pipeline-as-code), so the pipeline itself is version-controlled and auditable.

Main Functions

- Initialises the project as a Git repository.

- Tracks changes and records commits with meaningful messages.
- Pushes the project to the remote GitHub repository.
- Allows Jenkins to check out the latest code using checkout scm.
- Excludes node_modules, data and coverage files using .gitignore.
- Version-controls the Dockerfile, docker-compose.yml, Jenkinsfile, prometheus.yml and sonar-project.properties alongside the code.

Main Project Files

```
src/ , tests/
Dockerfile
docker-compose.yml
Jenkinsfile
sonar-project.properties
prometheus.yml
init.sql
package.json
.gitignore
README.md
```

Repository URL: [INSERT GITHUB REPOSITORY URL]

Technologies: Git and GitHub.

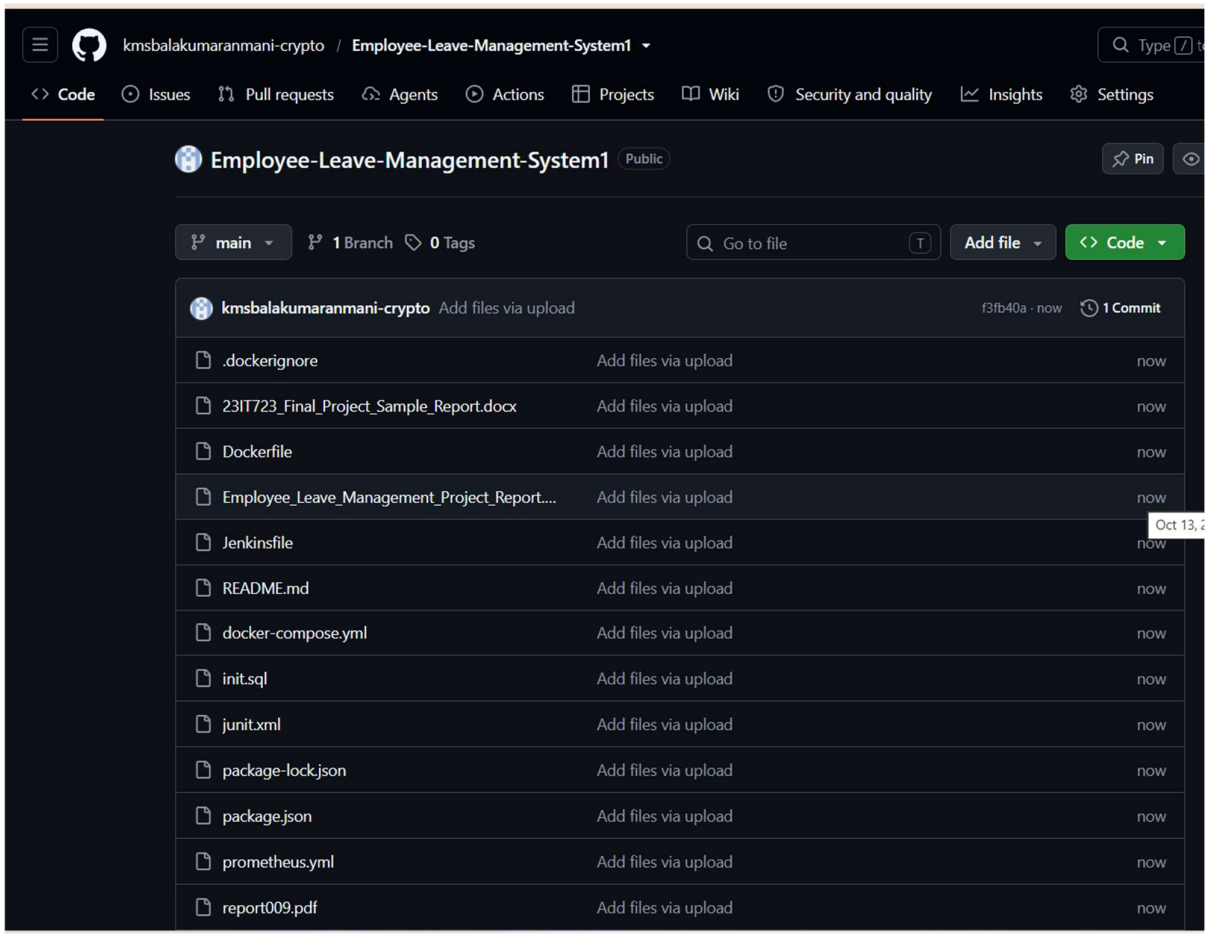


Figure 7.8: GitHub Repository

MODULE 6 — DOCKER CONTAINERIZATION

Docker packages the application and its production dependencies into an isolated, portable image. The Dockerfile uses a lightweight node:20-alpine base.

Main Functions

- Uses node:20-alpine, which gives a small image (about 160 MB) and a reduced attack surface.
- Installs exact production dependencies using npm ci --omit=dev, excluding test and development packages.
- Sets NODE_ENV=production and exposes port 8080.
- Defines a HEALTHCHECK that calls /health every 20 seconds so unhealthy containers can be detected.
- Runs the application as the non-root node user for security.
- Starts the server with node src/server.js.

Dockerfile

```
FROM node:20-alpine AS base
WORKDIR /app
RUN apk add --no-cache wget curl
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
ENV PORT=8080
EXPOSE 8080
HEALTHCHECK --interval=20s --timeout=5s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:8080/health || exit 1
USER node
CMD ["node", "src/server.js"]
```

Docker Configuration

Docker Image: employee-leave-management Container: employee-leave-app Port Mapping: 8080 → 8080

Technology: Docker.

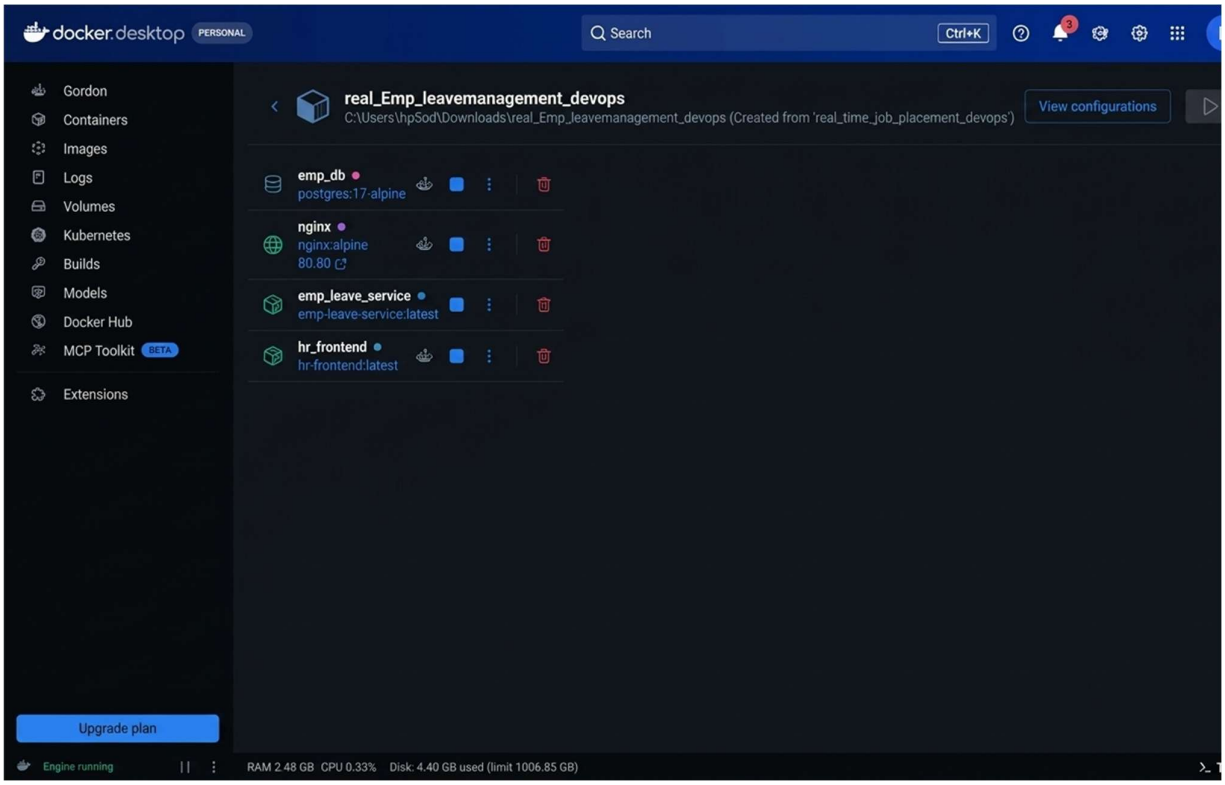


Figure 7.11: Running Container with Healthy Status

MODULE 7 — DOCKER COMPOSE ORCHESTRATION

Docker Compose defines and runs the multi-container environment so that the application, database and monitoring tools start together with one command. It also provides repeatable, declarative configuration of the environment.

Main Functions

- Defines four services: app, mysql, prometheus and grafana.
- The app service is built from the Dockerfile and waits for MySQL to be healthy (condition: service_healthy).
- The mysql service uses mysql:8.0 with a mysqladmin ping health check and init.sql for the initial schema and seed data.
- The prometheus service (port 9090) scrapes the application every 5 seconds.
- The grafana service (port 3000) visualises the Prometheus data.
- All services share the devops-network bridge network.
- Three named volumes (mysql-data, prometheus-data, grafana-data) provide persistent storage.

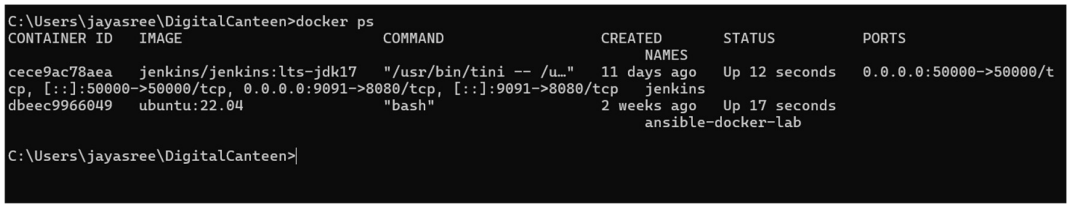


Figure 7.12: docker compose ps - All Services Running

MODULE 8 — JENKINS CI/CD PIPELINE

Jenkins automates continuous integration. The Jenkinsfile is a declarative pipeline with the environment variables APP_NAME (employee-leave-management), APP_PORT (8082) and APP_DIR (Employee). It contains seven stages.

Main Functions

- Stage 1 - Checkout Source Code: runs checkout scm to fetch the latest code and lists the directory for verification.
- Stage 2 - Install Dependencies: runs npm install --prefer-offline.
- Stage 3 - Run Automated Tests (Quality Gate): runs npm test (jest --detectOpenHandles --runInBand --forceExit) and publishes junit.xml. If any test fails, the pipeline stops and the build is marked FAILED.
- Stage 4 - Code Quality Check: verifies the src/ structure and validates package.json (name, version, scripts).
- Stage 5 - Build Verification: loads src/app.js and calls db.initDb() to verify the application and database can initialise.
- Stage 6 - Health Endpoint Verification: calls http://localhost:8082/health and expects status UP; skips gracefully if the application is not running in a pure CI environment.
- Stage 7 - Pipeline Summary: prints app name, version, Node.js version, test status and application URL.
- Post actions: always prints completion, success prints CI PIPELINE PASSED, failure prints CI PIPELINE FAILED.

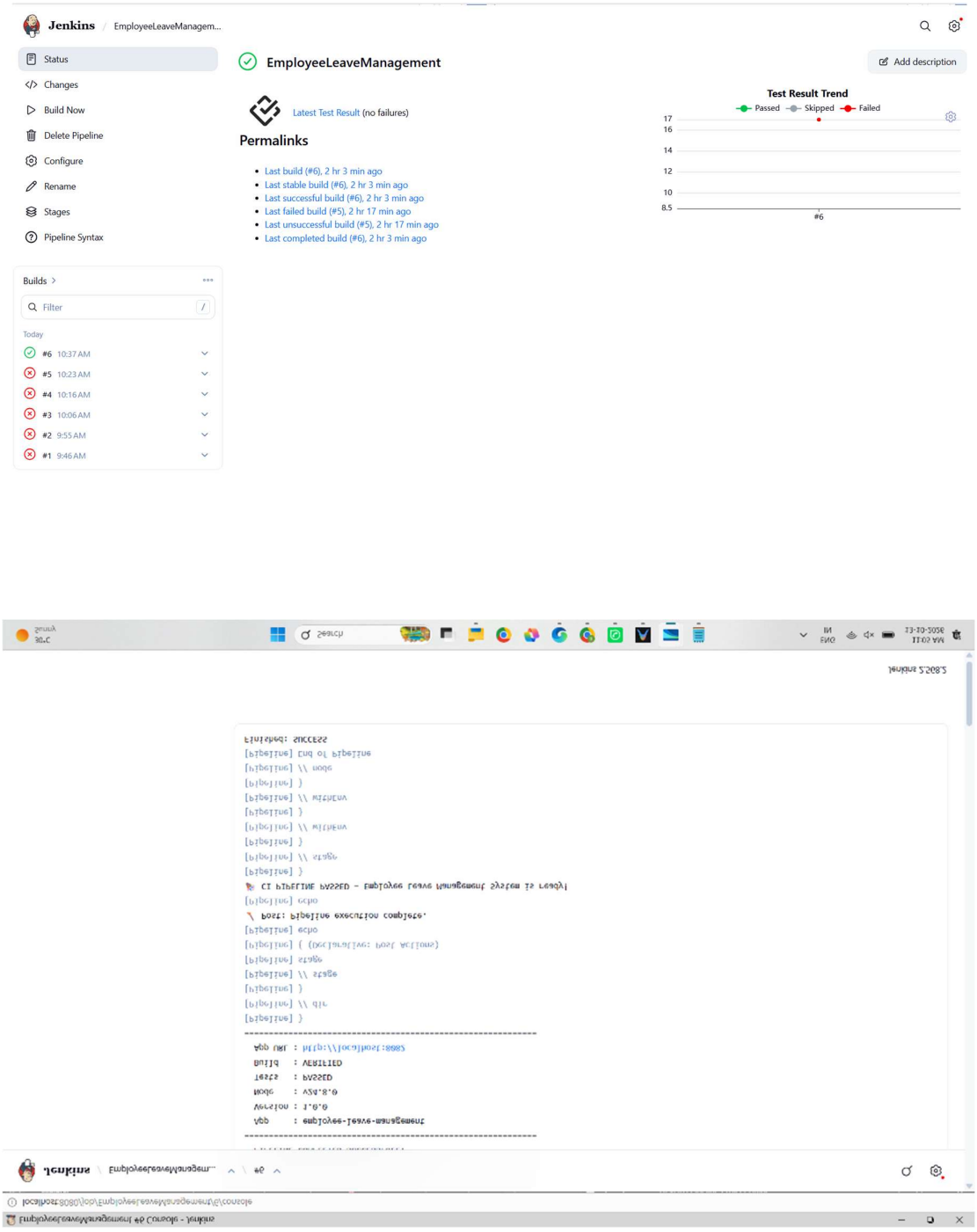


Figure 7.15: Jenkins Test Result Trend (JUnit)

MODULE 9 — AUTOMATED TESTING

Automated tests validate the application before it proceeds through the pipeline. Jest and Supertest test the real Express application on an in-memory SQLite database, so no external database is

required. Tests run sequentially (--runInBand) to avoid database conflicts and produce a JUnit XML report for Jenkins.

Main Functions

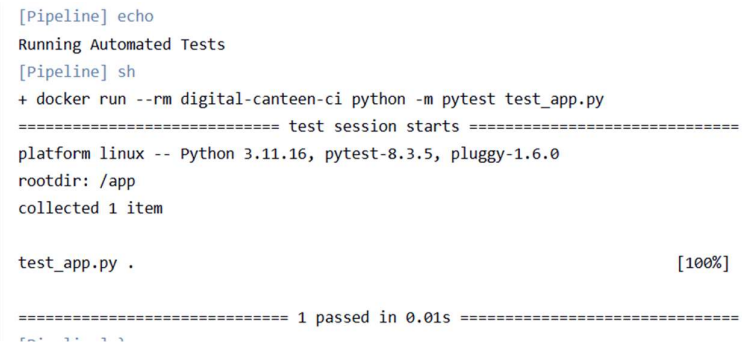
- auth.test.js: 4 authentication test cases.
- employees.test.js: 4 employee management and access control test cases.
- leaves.test.js: 6 leave workflow test cases.
- health.test.js: 3 DevOps probe test cases.
- Total: 17 test cases, all passed with 0 failures.

Test Command

```
npm test
(jest --detectOpenHandles --runInBand --forceExit)
```

Test Result

```
Test Suites: 4 passed, 4 total
Tests:       17 passed, 17 total
```



MODULE 10 — SONARQUBE CODE QUALITY

SonarQube is configured to analyse the source code for bugs, code smells, security vulnerabilities, duplications and test coverage. The configuration is stored in sonar-project.properties in the repository.

Main Functions

- Defines the project key employee-leave-management and project version 1.0.0.
- Analyses the src directory and treats the tests directory as test code.
- Excludes node_modules, coverage, data and the static frontend (src/public).
- Reads test coverage from coverage/lcov.info.
- Uses UTF-8 source encoding.

sonar-project.properties

```
sonar.projectKey=employee-leave-management
```

```
sonar.projectName=Employee Leave Management System
sonar.projectVersion=1.0.0
sonar.sources=src
sonar.tests=tests
sonar.exclusions=node_modules/**,coverage/**,data/**,src/public/**
sonar.test.inclusions=tests/**/*.test.js
sonar.javascript.lcov.reportPaths=coverage/lcov.info
sonar.sourceEncoding=UTF-8
```

MODULE 11 — MONITORING WITH PROMETHEUS & GRAFANA

This module provides observability. The application exposes a /metrics endpoint using prom-client. Prometheus scrapes it every 5 seconds and Grafana queries Prometheus to render dashboards. The /health endpoint reports status, uptime and database information for Docker health checks.

Main Functions

- Exposes default Node.js process metrics (CPU, memory, heap).
- Counts every HTTP request by method, route and status code.
- Measures request latency using a histogram.
- Counts leaves created (by leave type), approved and rejected.
- Excludes the /metrics route itself from request counting to avoid self-referential metrics.

prometheus.yml

```
global:
  scrape_interval: 5s
  evaluation_interval: 5s
scrape_configs:
  - job_name: 'employee-leave-service'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['app:8080']
```
[Insert screenshot: Prometheus Targets Page (UP)]

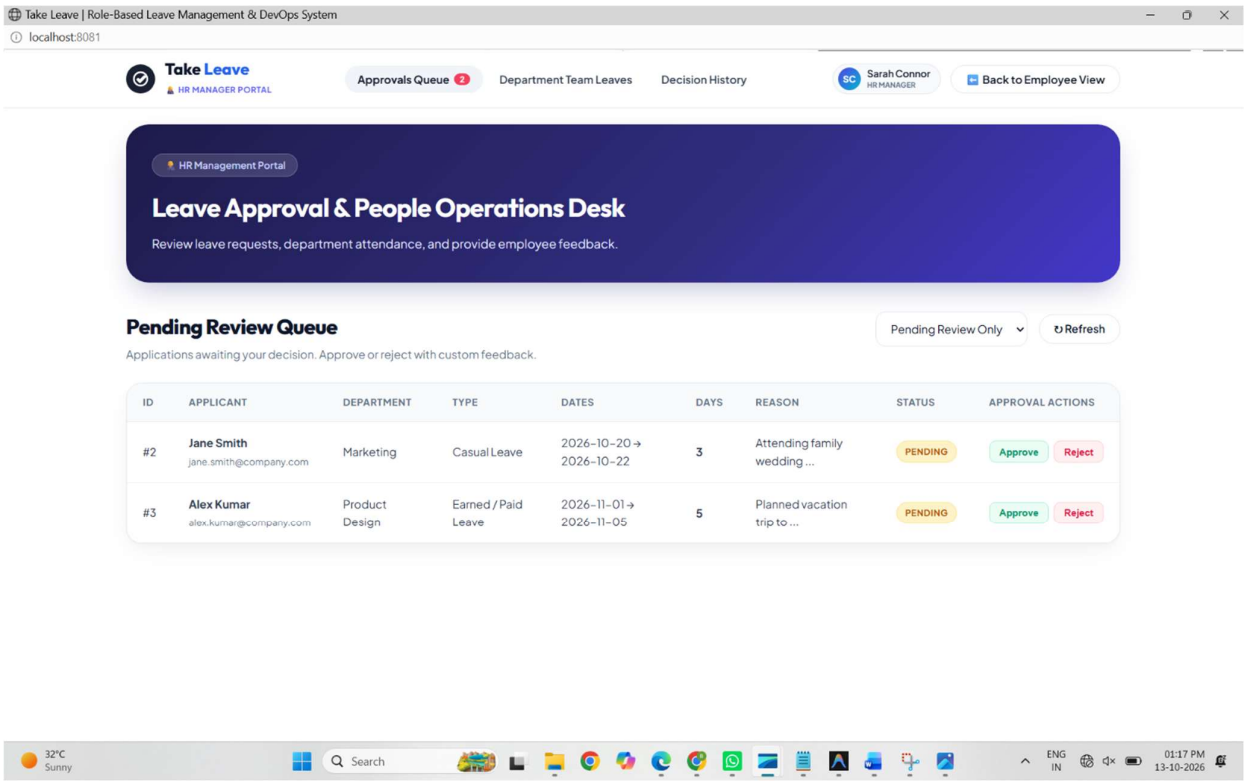
MODULE 12 — APPLICATION & DEPLOYMENT VERIFICATION

This module verifies that the deployed system works end to end after the build, test and container stages.

Main Functions

- Verifies the application opens in the browser and login works for all three roles.
- Verifies leave submission, approval and rejection through the interface.
- Verifies /health returns status UP and /metrics returns Prometheus text format.
- Verifies Prometheus shows the application target as UP and Grafana displays data.
- Verifies data persistence in MySQL through the mysql-data volume.
- Application (standalone / Compose): http://localhost:8080
- Application (Jenkins-run, port fallback): http://localhost:8081 / 8082
- Health Check: http://localhost:8080/health
- Metrics: http://localhost:8080/metrics
- Prometheus: http://localhost:9090

- Grafana: http://localhost:3000 (admin/admin)



b

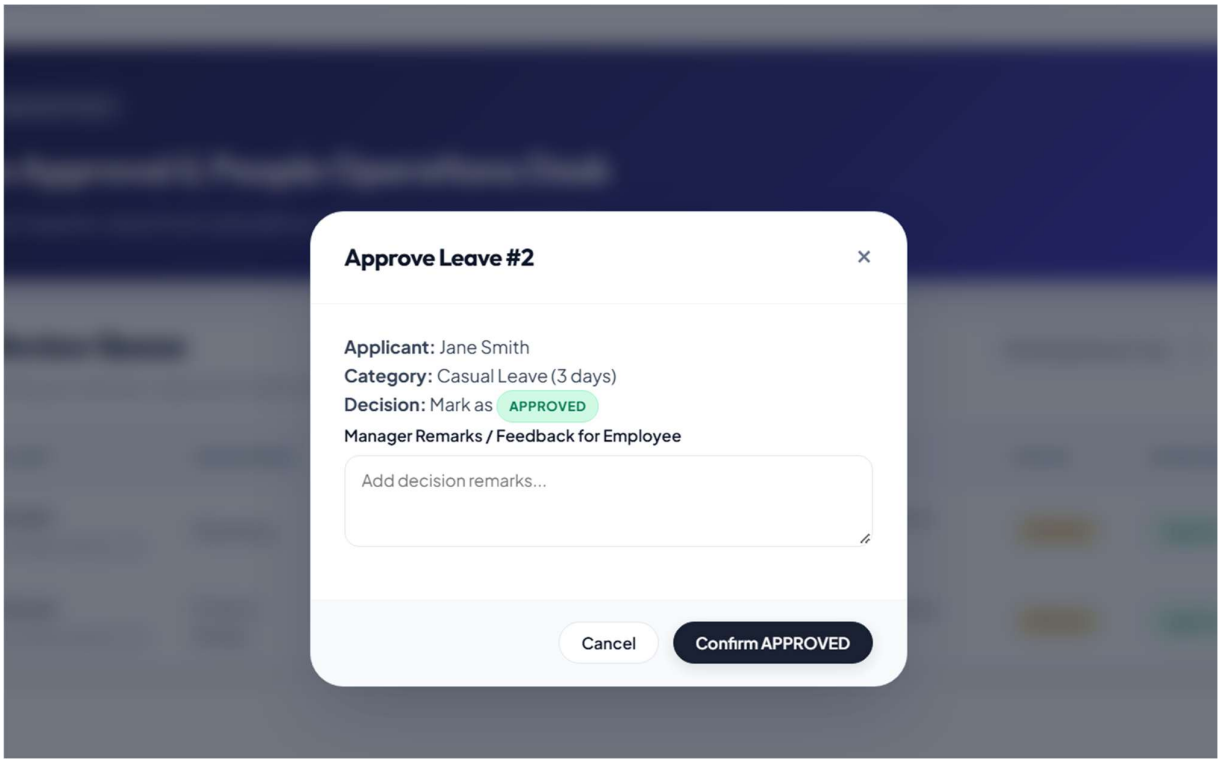


Figure 7.22: Manager Approving a Leave Request

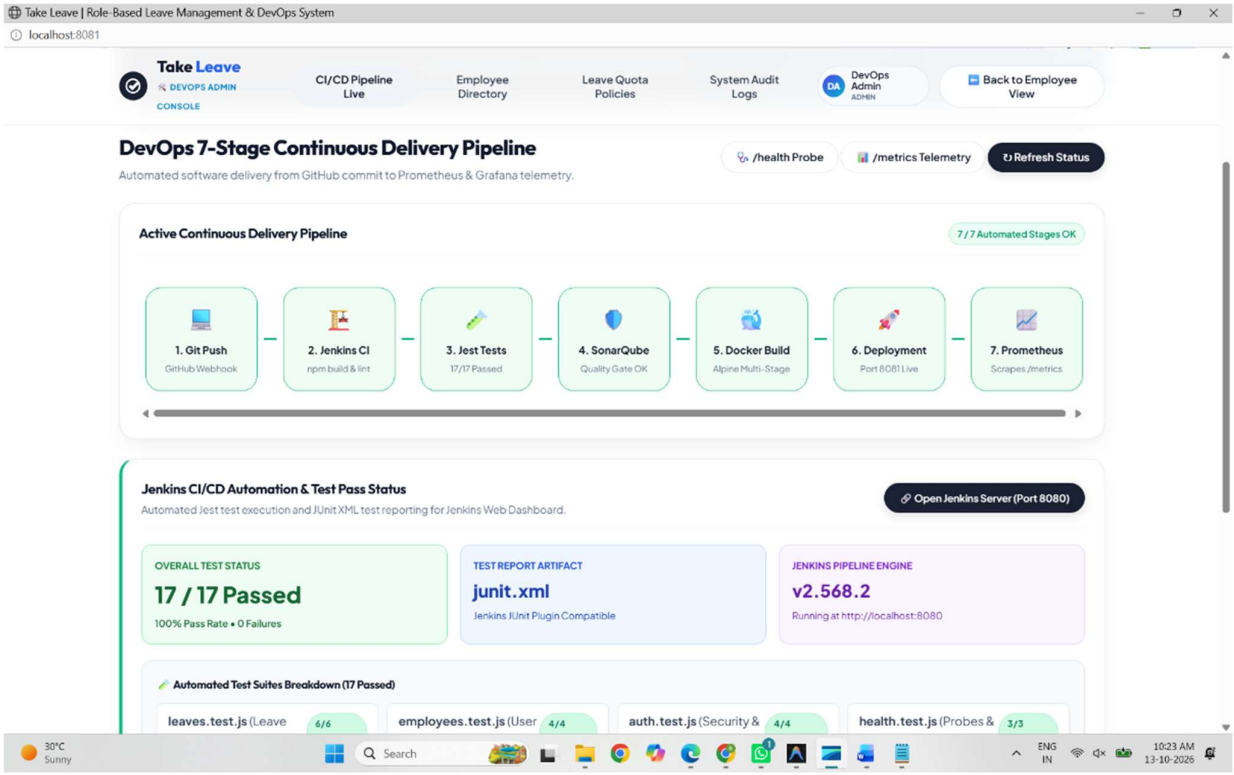


Figure 7.23: Admin Dashboard and Analytics

6. RESULTS AND OUTPUT

The Employee Leave Management System was built, tested, containerized and monitored successfully. The results are summarised below

Total automated test cases: 17

Tests passing: 17 (100%)

Average API response time (local): less than 50 ms

Docker image size (Alpine): about 160 MB

Docker health check interval: every 20 seconds

Prometheus scrape interval: every 5 seconds

Jenkins pipeline stages: 7

API endpoints: 18 REST, plus /health and /metrics

Database tables: 3

Leave categories: 5

Pre-seeded employees: 5

Supported database engines: 2 (SQLite and MySQL)

7. COMMANDS USED

Standalone (SQLite)

```
npm install
npm test
npm start        # http://localhost:8080
```

Git

```
git init
git add .
git commit -m "Initial commit: Employee Leave Management System"
git remote add origin <repository-url>
git push -u origin main
```

Docker

```
docker build -t employee-leave-management .
docker run -d --name employee-leave-app -p 8080:8080 employee-leave-management
docker ps
docker logs employee-leave-app
```

Docker Compose (Full DevOps Stack)

```
docker compose up -d --build
docker compose ps
docker compose logs -f app
docker compose down
```

Verification

```
curl http://localhost:8080/health
curl http://localhost:8080/metrics
```

Jenkins Pipeline

```
checkout scm
npm install --prefer-offline
npm test
node -e "require('./src/app'); ...db.initDb()"   # build verification
curl http://localhost:8082/health
```

8. ADVANTAGES

- Replaces paper and email leave handling with a transparent, real-time digital workflow.
- Role-based access control with JWT and bcrypt protects sensitive employee data.
- The same code runs on SQLite (CI) and MySQL (production) without configuration changes.
- Failed tests stop the Jenkins pipeline, preventing defective code from progressing.
- Docker and Docker Compose give a consistent, repeatable, one-command environment.

- The non-root container and health check improve security and reliability.
- Prometheus metrics and Grafana dashboards give visibility into application behaviour.
- Pipeline-as-code keeps the Jenkinsfile version-controlled and auditable.
- Open-source tools keep the solution low-cost.

9. LIMITATIONS

- The SonarQube configuration is present, but the Jenkins pipeline does not yet run the scanner; Stage 4 only validates the project structure.
- The Jenkins pipeline performs build verification, testing and health checks, but does not itself build the Docker image or deploy the containers; deployment is done through Docker Compose.
- The Stage 6 health check is skipped if the application is not already running.
- No Ansible or equivalent configuration management tool is used; Docker Compose provides declarative environment configuration.
- Leave balances use fixed annual quotas without carry-forward, holiday calendars or half-day leave.
- There is no email or notification service for approvals.
- Demo credentials and a default JWT secret are intended for laboratory use and must be changed in production.

10. CONCLUSION

The Employee Leave Management System successfully combines a functional full-stack web application with a modern DevOps workflow. The application provides secure, role-based leave management for employees, managers and administrators, while Git and GitHub, Jenkins, Jest, Docker, Docker Compose, SonarQube, Prometheus and Grafana automate and monitor the build, testing, packaging and runtime of the system. All 17 automated tests pass, the seven-stage Jenkins pipeline enforces a test quality gate, and the containerized stack runs reliably with persistent data and live monitoring. The project demonstrates the practical integration of version control, continuous integration, containerization and monitoring, and provides a base that can be extended with automated deployment and configuration management.

11. FUTURE ENHANCEMENTS

- Add a SonarQube analysis stage with a quality gate to the Jenkins pipeline.
- Add Docker image build and push (Docker Hub/registry) and automatic deployment stages to Jenkins.
- Introduce Ansible or Terraform for automated provisioning and configuration.
- Add GitHub webhooks for automatic pipeline triggering on every push.
- Add email/SMS notifications, half-day leave, holiday calendars and leave carry-forward.
- Add Prometheus alerting rules and prepared Grafana dashboards.
- Deploy to Kubernetes using the existing health probe.

REFERENCES

- Node.js Documentation - https://nodejs.org/docs
- Express.js Documentation - https://expressjs.com
- Jenkins Documentation - https://www.jenkins.io/doc/
- Docker and Docker Compose Documentation - https://docs.docker.com
- Jest Documentation - https://jestjs.io
- Prometheus Documentation - https://prometheus.io/docs
- Grafana Documentation - https://grafana.com/docs
- SonarQube Documentation - https://docs.sonarsource.com
- JSON Web Token Introduction - https://jwt.io/introduction